

MLOps Fundamentals Lab Guide

Introduction to MLOps · ML Lifecycle · Environment Setup · Model Serialization & Configuration Management (AWS Console)

Prepared for Hands-On Practice

[Module 1: Introduction to MLOps](#)

[1.1 What is MLOps?](#)

[1.2 Why MLOps Matters](#)

[Lab 1.1 — Explore the AWS Console and Set Up Your MLOps Workspace](#)

[Module 2: The ML Lifecycle](#)

[2.1 Stages of the ML Lifecycle](#)

[Lab 2.1 — Launch a SageMaker Notebook Instance for the ML Lifecycle](#)

[Lab 2.2 — Run a Minimal ML Lifecycle Example](#)

[Module 3: Environment Setup](#)

[3.1 Why Environment Consistency Matters](#)

[Lab 3.1 — Create a Reproducible Environment Definition](#)

[Lab 3.2 — Explore SageMaker's Managed Training Environments](#)

[Module 4: Model Serialization & Configuration Management](#)

[4.1 What is Model Serialization?](#)

[4.2 What is Configuration Management \(in MLOps\)?](#)

[Lab 4.1 — Serialize a Trained Model and Store it in S3](#)

[Lab 4.2 — Reload the Serialized Model and Verify Predictions](#)

[Lab 4.3 — Externalize Configuration Using AWS Systems Manager Parameter Store](#)

[Cleanup \(Important — Avoid Ongoing Charges\)](#)

[Summary](#)

Module 1: Introduction to MLOps

1.1 What is MLOps?

MLOps (Machine Learning Operations) is the set of practices that combines Machine Learning, DevOps, and Data Engineering to reliably and efficiently deploy and maintain ML models in production. It addresses the gap between building a model in a notebook and running that model safely, reproducibly, and at scale for real users.

Core pillars of MLOps:

- **Reproducibility** — the same code, data, and configuration should always produce the same model
- **Automation** — training, testing, and deployment pipelines run with minimal manual intervention
- **Monitoring** — tracking model performance, data drift, and infrastructure health after deployment
- **Collaboration** — data scientists, ML engineers, and operations teams working from a shared, versioned source of truth
- **Governance** — auditability, access control, and compliance for regulated environments

1.2 Why MLOps Matters

A model that works in a Jupyter notebook is not the same as a model running reliably in production. Without MLOps discipline, teams typically face:

- Models that can't be reproduced because the training environment or data snapshot is lost
- No visibility into which model version is currently serving predictions
- Manual, error-prone deployment steps

- Silent model performance degradation (drift) going undetected

Lab 1.1 — Explore the AWS Console and Set Up Your MLOps Workspace

Objective: Get comfortable navigating the AWS Console areas you will use throughout these labs.

Prerequisites: An AWS account with console access (IAM user or role with permissions for SageMaker, S3, and IAM).

Steps:

1. Sign in to the **AWS Management Console** at <https://console.aws.amazon.com>.
2. In the top search bar, type **“S3”** and open the S3 console. This is where you will store datasets, model artifacts, and configuration files.
3. Click **Create bucket**.
 - Bucket name: `mlops-labs-<your-initials>-<random-number>` (bucket names must be globally unique)
 - Region: choose a region close to you, e.g., `us-east-1`
 - Leave other settings as default and click **Create bucket**.
4. Inside your new bucket, create three folders (using **Create folder**):
 - `raw-data/`
 - `model-artifacts/`
 - `configs/`
5. In the top search bar, type **“SageMaker”** and open the **Amazon SageMaker** console. This is the primary AWS service you will use for the ML lifecycle labs.
6. On the SageMaker left-hand navigation pane, note the sections you’ll use later: **Notebook instances / Studio, Training jobs, Models, Inference**.

Checkpoint: You should now have an S3 bucket with three folders, and you should be able to see the SageMaker console home page.

Module 2: The ML Lifecycle

2.1 Stages of the ML Lifecycle

Stage	Description
1. Problem Framing	Define the business problem and success metrics
2. Data Collection & Preparation	Gather, clean, and label data
3. Feature Engineering	Transform raw data into model-ready features
4. Model Training	Fit a model on training data
5. Model Evaluation	Validate performance against held-out data
6. Model Serialization	Save the trained model to a portable format
7. Deployment	Serve the model for predictions
8. Monitoring & Retraining	Track performance and refresh the model over time

Lab 2.1 — Launch a SageMaker Notebook Instance for the ML Lifecycle

Objective: Stand up a managed Jupyter environment in AWS to run a simple end-to-end ML lifecycle example.

Steps:

1. In the **SageMaker console**, go to **Notebook > Notebook instances**.
2. Click **Create notebook instance**.
 - Notebook instance name: `mlops-lab-notebook`
 - Notebook instance type: `ml.t3.medium` (low cost, sufficient for these labs)

- Under **Permissions and encryption**, choose **Create a new role**, and grant it access to your specific S3 bucket from Lab 1.1.
3. Click **Create notebook instance**. Status will show **Pending** for a few minutes, then change to **InService**.
 4. Once **InService**, click **Open Jupyter** (or **Open JupyterLab**).
 5. Create a new notebook using the **conda_python3** kernel.

Checkpoint: You have a running Jupyter notebook backed by a managed AWS instance.

Lab 2.2 — Run a Minimal ML Lifecycle Example

Objective: Walk through data prep, training, and evaluation in code, so you can see each lifecycle stage concretely before automating it.

Steps (run each cell in your SageMaker notebook):

```
# Step 1: Data Collection & Preparation
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

data = load_iris(as_frame=True)
df = data.frame
X_train, X_test, y_train, y_test = train_test_split(
    df[data.feature_names], df['target'], test_size=0.2, random_state=42
)
```

```
# Step 2: Model Training
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
```

```
# Step 3: Model Evaluation
from sklearn.metrics import accuracy_score

predictions = model.predict(X_test)
accuracy = accuracy_score(y_test, predictions)
print(f"Model accuracy: {accuracy:.4f}")
```

```
# Step 4: Upload evaluation results to S3 for record-keeping
import boto3
import json

s3 = boto3.client("s3")
bucket_name = "mlops-labs-<your-initials>-<random-number>" # replace with your bucket

results = {"accuracy": accuracy, "model": "RandomForestClassifier", "n_estimators": 100}
s3.put_object(
    Bucket=bucket_name,
    Key="configs/lab2_evaluation_results.json",
    Body=json.dumps(results)
)
print("Results uploaded to S3.")
```

Checkpoint: In the S3 console, refresh your `configs/` folder and confirm `lab2_evaluation_results.json` was uploaded.

Module 3: Environment Setup

3.1 Why Environment Consistency Matters

One of the most common causes of “it worked on my machine” failures is environment drift — different library versions between training and serving. MLOps addresses this through:

- **Managed notebook/training environments** (e.g., SageMaker-managed instances with fixed images)
- **Requirements files** that pin exact package versions
- **Container images** that package code, dependencies, and runtime together

Lab 3.1 — Create a Reproducible Environment Definition

Objective: Capture your notebook’s Python environment as a `requirements.txt` file and store it in S3 for reproducibility.

Steps:

1. In your SageMaker notebook from Lab 2.1, open a new terminal: **File > New > Terminal** (JupyterLab) or use a notebook cell with `!` prefix.
2. Run the following to capture installed packages:

```
!pip freeze > requirements.txt
!cat requirements.txt
```

3. Upload this file to S3 directly from the notebook:

```
s3.upload_file("requirements.txt", bucket_name, "configs/requirements.txt")
print("Environment definition uploaded.")
```

4. In the **S3 console**, navigate to your bucket’s `configs/` folder and confirm `requirements.txt` is present.

Checkpoint: Anyone on your team can now download `requirements.txt` from S3 and recreate your exact Python environment with `pip install -r requirements.txt`.

Lab 3.2 — Explore SageMaker’s Managed Training Environments

Objective: See how SageMaker uses pre-built Docker images to guarantee environment consistency between your notebook and actual training jobs, without you needing to manage Docker directly.

Steps:

1. In the SageMaker console, go to **Training > Training jobs**.
2. Click **Create training job**.
3. Under **Algorithm source**, select **Amazon SageMaker built-in algorithm**, then choose **XGBoost** from the list.
4. Note the **container image URI** that SageMaker auto-populates — this is the exact, versioned environment your training job will run in.
5. You do not need to complete this training job for this lab — click **Cancel** once you’ve located and noted the image URI field. This confirms you can locate where AWS defines the training environment.

Checkpoint: You can identify where SageMaker specifies the managed environment/image used for a training job.

Module 4: Model Serialization & Configuration Management

4.1 What is Model Serialization?

Model serialization is the process of converting a trained, in-memory model object into a file format that can be saved, transferred, and later reloaded for inference — without retraining. Common formats include:

Format	Typical Use
--------	-------------

<code>.pkl</code> (pickle) / <code>joblib</code>	scikit-learn models
<code>.pt</code> / <code>.pth</code>	PyTorch models
<code>.h5</code> / SavedModel	TensorFlow/Keras models
ONNX	Cross-framework portable format

4.2 What is Configuration Management (in MLOps)?

Configuration management means externalizing settings — hyperparameters, file paths, environment variables, resource sizes — from code, so the same code can run in different contexts (dev/test/prod) without modification, and every run is traceable to a specific configuration.

Lab 4.1 — Serialize a Trained Model and Store it in S3

Objective: Save the model trained in Lab 2.2 to disk using `joblib`, then upload it to your S3 `model-artifacts/` folder — the standard MLOps pattern for versioned model storage.

Steps (continue in the same SageMaker notebook):

```
# Step 1: Serialize the model
import joblib

joblib.dump(model, "iris_model.joblib")
print("Model serialized to iris_model.joblib")
```

```
# Step 2: Upload the serialized model to S3
s3.upload_file(
    "iris_model.joblib",
    bucket_name,
    "model-artifacts/iris_model_v1.joblib"
)
print("Model uploaded to S3 model-artifacts/ folder.")
```

3. In the **S3 console**, open your bucket's `model-artifacts/` folder and confirm `iris_model_v1.joblib` is listed. Click on the object to view its **Version ID**, **Size**, and **Last modified** — this is your basic model artifact record.

Checkpoint: You have a versioned, portable model file stored independently of your notebook.

Lab 4.2 — Reload the Serialized Model and Verify Predictions

Objective: Confirm the serialized model works correctly when loaded fresh — simulating what a separate serving process would do.

Steps:

```
# Step 1: Download the model back from S3 (simulating a fresh environment)
s3.download_file(
    bucket_name,
    "model-artifacts/iris_model_v1.joblib",
    "reloaded_model.joblib"
)

# Step 2: Load it and run a prediction
reloaded_model = joblib.load("reloaded_model.joblib")
sample_prediction = reloaded_model.predict(X_test.iloc[[0]])
print(f"Prediction from reloaded model: {sample_prediction}")
```

Checkpoint: The reloaded model produces the same prediction as the original in-memory model — confirming successful serialization and reload.

Lab 4.3 — Externalize Configuration Using AWS Systems Manager Parameter Store

Objective: Move hyperparameters out of your code and into a managed configuration store, so they can be changed without editing or redeploying code.

Steps:

1. In the AWS Console search bar, type **“Systems Manager”** and open it.
2. In the left-hand navigation, go to **Application Management > Parameter Store**.
3. Click **Create parameter**.
 - Name: `/mlops-lab/model/n_estimators`
 - Tier: Standard
 - Type: String
 - Value: `100`
4. Click **Create parameter**. Repeat to add a second parameter:
 - Name: `/mlops-lab/model/random_state`
 - Value: `42`
5. Back in your SageMaker notebook, retrieve these parameters at runtime instead of hardcoding them:

```
import boto3

ssm = boto3.client("ssm")

n_estimators = int(ssm.get_parameter(Name="/mlops-lab/model/n_estimators")["Parameter"]["Value"])
random_state = int(ssm.get_parameter(Name="/mlops-lab/model/random_state")["Parameter"]["Value"])

print(f"Loaded config -> n_estimators: {n_estimators}, random_state: {random_state}")

# Reuse in model training
from sklearn.ensemble import RandomForestClassifier
model_v2 = RandomForestClassifier(n_estimators=n_estimators, random_state=random_state)
model_v2.fit(X_train, y_train)
```

Checkpoint: You changed a model hyperparameter in the AWS Console (not in code) and had your notebook pick it up automatically — this is the core idea behind configuration management in MLOps.

Cleanup (Important — Avoid Ongoing Charges)

To avoid unnecessary AWS billing after completing these labs:

1. **SageMaker Notebook Instance:** Go to SageMaker console > Notebook instances > select `mlops-lab-notebook` > **Actions > Stop**, then **Actions > Delete** once stopped.
2. **S3 Bucket:** Go to S3 console > select your bucket > **Empty** (delete all objects) > then **Delete bucket**.
3. **Systems Manager Parameters:** Go to Systems Manager > Parameter Store > select both `/mlops-lab/...` parameters > **Delete**.
4. **IAM Role:** Go to IAM console > Roles > find the role auto-created for your notebook in Lab 2.1 > **Delete** if no longer needed.

Summary

Module	Key Skill Practiced
Introduction to MLOps	AWS Console navigation, S3 bucket structure for ML artifacts
ML Lifecycle	End-to-end data prep → train → evaluate flow in SageMaker
Environment Setup	Capturing and storing reproducible environment definitions

Model Serialization & Config Management	Saving/reloading models, externalizing config via Parameter Store
---	--